# DATABASE DESIGN DOCUMENT

## Database Purpose

The TrackWIL database stores everything the application needs to track Work-Integrated Learning logbooks: the accounts of students, coordinators and lecturers; each student's daily logbook entries with their hours, evidence file name and verification status; and the one-time verification links that workplace mentors use to approve or reject entries.

The database must support these queries efficiently and reliably:

- Sign-in: find a user by email address (case-insensitive).
- Student logbook: list one student's entries, newest first, and total their verified hours.
- Verification: find a link by its token, check that it is unused and unexpired, and record the mentor's decision on the link and the entry together.
- Monitoring dashboard: for every student, count entries and pending verifications, total verified hours and find the latest verification date.
- Evidence download: find the entry that owns an evidence file to check who may open it.

SQLite was chosen because it is a full relational database (keys, constraints and transactions) that runs inside the application through Node.js's built-in node:sqlite module, so no database server has to be installed. The database is a single file, data/trackwil.db, which is created and seeded automatically when the application first starts.

## Entity-Relationship Diagram

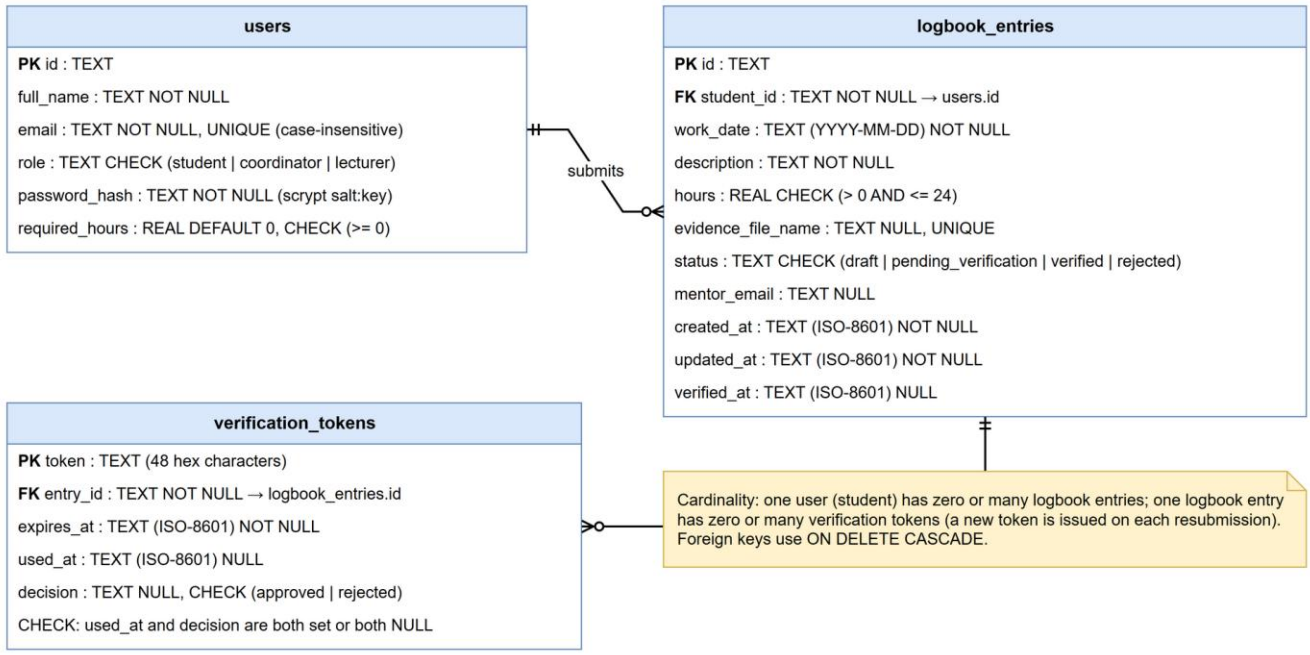



Figure 1: Entity-relationship diagram

Table 1: Relationships and cardinalities

| Relationship | Cardinality | Implemented by | Meaning |
|---|---|---|---|
| users to logbook_entries | One to zero-or-many | logbook_entries.student_id references users.id | A student can have any number of entries; each entry belongs to exactly one student |
| logbook_entries to verification_tokens | One to zero-or-many | verification_tokens.entry_id references logbook_entries.id | An entry gets a new link each time it is submitted (including resubmission after rejection); each link belongs to exactly one entry |

Mentors are not stored as a separate entity because they have no account in version 1.0. The mentor is identified by the email address stored on each entry, and each verification is recorded on the token that the mentor used.

## Data Dictionary

All text values are stored as TEXT and all numbers as REAL, which are SQLite's storage classes. Dates and times are stored as ISO-8601 text in UTC (for example 2026-10-22T08:00:00.000Z), which sorts correctly as text.

Table 2: users

| Column | Type | Size / format | Null | Key | Constraint and description |
|---|---|---|---|---|---|
| id | TEXT | Up to 44 characters (e.g. student-<UUID>) | No | PK | Unique account identifier |
| full_name | TEXT | Up to about 120 characters | No | | The user's full name, shown in headers and the dashboard |
| email | TEXT | Up to about 150 characters | No | Unique | Sign-in name; UNIQUE with COLLATE NOCASE so letter case does not matter |
| role | TEXT | student, coordinator or lecturer | No | | CHECK limits the value to the three roles |
| password_hash | TEXT | 32-character salt, a colon and a 128-character key | No | | scrypt hash of the password; the password itself is never stored |
| required_hours | REAL | 0 or more | No | | Hours the student must complete (default 200 for students, 0 for staff); CHECK (>= 0) |

Table 3: logbook_entries

| Column | Type | Size / format | Null | Key | Constraint and description |
|---|---|---|---|---|---|
| id | TEXT | UUID (36 characters) | No | PK | Unique entry identifier |
| student_id | TEXT | As users.id | No | FK | References users(id) ON DELETE CASCADE; indexed with work_date |
| work_date | TEXT | YYYY-MM-DD | No | | The day the work was done |
| description | TEXT | Free text | No | | What the student did |
| hours | REAL | More than 0, at most 24 | No | | CHECK (hours > 0 AND hours <= 24) |
| evidence_file_name | TEXT | UUID plus extension | Yes | Unique | Name of the stored evidence file, or NULL if none |
| status | TEXT | draft, pending_verification, verified, rejected | No | | CHECK limits the value; default draft |
| mentor_email | TEXT | Email address | Yes | | The workplace mentor for this entry; required before submission |
| created_at | TEXT | ISO-8601 UTC | No | | When the entry was created |
| updated_at | TEXT | ISO-8601 UTC | No | | When the entry last changed |
| verified_at | TEXT | ISO-8601 UTC | Yes | | When the mentor approved the entry |

Table 4: verification_tokens

| Column | Type | Size / format | Null | Key | Constraint and description |
|---|---|---|---|---|---|
| token | TEXT | 48 hexadecimal characters | No | PK | Random value from 24 bytes (192 bits); appears in the link /verify/<token> |
| entry_id | TEXT | As logbook_entries.id | No | FK | References logbook_entries(id) ON DELETE CASCADE; indexed |
| expires_at | TEXT | ISO-8601 UTC | No | | 7 days after the link was created |
| used_at | TEXT | ISO-8601 UTC | Yes | | When the mentor used the link; NULL while unused |
| decision | TEXT | approved or rejected | Yes | | The mentor's decision; CHECK requires used_at and decision to be both set or both NULL |

## Normalisation

**First normal form (1NF).** Every column holds one value and every table has a primary key. For example, each day of work is its own row in logbook_entries rather than a list of days stored in one column of users.

**Second normal form (2NF).** Every table has a single-column primary key, so no column can depend on only part of a key.

**Third normal form (3NF).** No non-key column depends on another non-key column. A student's name and required hours are stored once in users, not repeated in every entry, so changing a name updates every screen at once (no update anomaly). Each token row holds only facts about that link.

**Calculated values are not stored.** A student's verified hours and their On track, Awaiting verification or Overdue status are calculated from the entries every time the dashboard is shown. Storing them would create duplicate data that could become wrong when an entry changes.

**Change from the prototype.** The first prototype stored each collection as a JSON file and rewrote the whole file on every change, with no keys or constraints. Moving to SQLite added enforced primary and foreign keys, CHECK constraints and transactions without changing the shape of the data.

## Security and Integrity

- **Constraints:** primary keys, UNIQUE (email, evidence file name), foreign keys with PRAGMA foreign_keys = ON, and CHECK constraints for roles, statuses, hours and the token decision.
- **Transactions:** submitting an entry (insert token and set Pending) and recording a mentor decision (mark token used and update the entry) each run in a single transaction, so a failure cannot leave a half-finished change. The token is marked used only WHERE used_at IS NULL, so a link cannot be used twice.
- **Validation:** every value is validated in the Server Action before it reaches the database; the constraints are a second line of defence.
- **SQL injection:** every query uses prepared statements with ? placeholders; user input is never joined into SQL text.

- **Access control:** only the repository module runs SQL, and it is called only from server code after the user's role and ownership have been checked.
- **Sensitive data:** passwords are stored only as salted scrypt hashes. Evidence files are stored outside the public web folder under random names and served only to their owner and to staff.
- **Backups and recovery:** the database is a single file that can be copied while the application is stopped. For the demonstration, npm run db:reset rebuilds it from the scripts.
- **Privacy:** only synthetic data is stored until written campus permission is granted (POPIA).

## SQL Scripts

Table 5: Database scripts in the repository

| Script | Location | Purpose |
|---|---|---|
| schema.sql | 05_Source_Code/trackwil/database/schema.sql | Creates the three tables, keys, constraints and indexes |
| seed.sql | 05_Source_Code/trackwil/database/seed.sql | Inserts synthetic demonstration users, entries and links, with dates relative to the current date |
| reset-db.mjs | 05_Source_Code/trackwil/scripts/reset-db.mjs | Deletes and rebuilds the database from the two scripts (npm run db:reset) |
| db.ts | 05_Source_Code/trackwil/src/lib/db.ts | Applies the schema on start-up and seeds an empty database |
| repository.ts | 05_Source_Code/trackwil/src/lib/repository.ts | All queries and transactions used by the application |